1)

```
module ct1(A, B, C, D, E, F, sel, out, out_bar);
  input A, B, C, D, E, F, sel;
  output out, out_bar;
  wire and1_out, xnor1_out, mux_out;

  assign and1_out = A & B & C;
  assign xnor1_out = ~(D ^ E ^ F);

  assign out = (sel == 0)? and1_out: xnor1_out;
  assign out_bar = ~out;
endmodule
```

2)

```
module ct2(A, B, C, D, sel, out, out_bar);
  input A, B, C, sel;
  input [2:0] D;
  output reg out, out_bar;
  reg and2_out, or1_out, xnor1_out;

  always@(*) begin
    and2_out = D[0] & D[1];
    or1_out  = and2_out | D[2];
    xnor1_out = ~(A ^ B ^ C);

    if(sel)
      out = xnor1_out;
    else
      out = or1_out;

    out_bar = ~out;
  end
endmodule
```

3)

```
module adder(A, B, C);
  input [3:0] A, B;
  output [3:0] C;

  assign C = A + B;

endmodule
```

3)

```
module decoder(A, D);
  input [1:0] A;
  output [3:0] D;

  assign D = (A == 0)? 4'b0001: (A == 1)? 4'b0010: (A == 2)? 4'b0100: 4'b1000;

  /* OR
  assign D[3] = (A == 3)? 1:0;
  assign D[2] = (A == 2)? 1:0;
  assign D[1] = (A == 1)? 1:0;
  assign D[0] = (A == 0)? 1:0;
  */
endmodule
```

5)

```
module q5_parity(A, out);
  input [7:0] A;
  output [8:0] out;

  assign out = {A, ^A}; // even parity generator

endmodule
```

6)

```
module segment_LED_display(A,B,opcode,enable,a,b,c,d,e,f,g);
 input [3:0] A,B;
 input [1:0] opcode;
 input enable;
 output reg a,b,c,d,e,f,g;

 wire [3:0] alu_out;
 reg [6:0] display_out;

 n_bit_alu #(4) alu1(.A(A),.B(B),.opcode(opcode),.result(alu_out));

 always @(*) begin
  if (!enable)
   display_out = 7'b0;
  else begin
   case(alu_out)
    4'h0 : display_out = 7'b111_1110;
    4'h1 : display_out = 7'b011_0000;
    4'h2 : display_out = 7'b110_1101;
    4'h3 : display_out = 7'b111_1001;
    4'h4 : display_out = 7'b011_0011;
    4'h5 : display_out = 7'b101_1011;
    4'h6 : display_out = 7'b101_1111;
    4'h7 : display_out = 7'b111_0000;
    4'h8 : display_out = 7'b111_1111;
    4'h9 : display_out = 7'b111_1011;
    4'hA : display_out = 7'b111_0111;
    4'hB : display_out = 7'b001_1111;
    4'hC : display_out = 7'b100_1110;
    4'hD : display_out = 7'b011_1101;
    4'hE : display_out = 7'b100_1111;
    4'hF : display_out = 7'b100_0111;
    default: display_out = 7'b0;
   endcase
  end
 end

 always @(*) begin
  a = display_out[6];
  b = display_out[5];
  c = display_out[4];
  d = display_out[3];
  e = display_out[2];
  f = display_out[1];
  g = display_out[0];
 end
endmodule
```